Security Optimization Assessment — Defender for Cloud Apps

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Security Optimization Assessment: Defender for Cloud Apps 4
- What you get . . . 4
- What this delivers . . . 4
- The standard deployment model . . . 4
- Boundary . . . 5
- Effort . . . 5

1. Solution Overview — Security Optimization Assessment: Defender for Cloud Apps 5
- What Defender for Cloud Apps is . . . 5
- Why this assessment matters . . . 5
- The four assessment domains and how they are scored . . . 6
- What is in scope . . . 6
- What is out of scope (and where it lives) . . . 6
- Connections and authentication used . . . 6
- The standard deployment model . . . 7

2. Requirements — Security Optimization Assessment: Defender for Cloud Apps 7
- R1. Defender for Cloud Apps is provisioned in the tenant . . . 7
- R2. Licensing for Defender for Cloud Apps (and app governance) . . . 7
- R3. Read-only Security Reader / Global Reader access . . . 8
- R4. Microsoft Graph read scopes consented . . . 8
- R5. Cloud discovery data source context . . . 8
- R6. A defined assessment scope and a secure output location . . . 8

Global Requirements — Assessments (category-level) 8
- G1. The workload/tenant being assessed . . . 9
- G2. Read-only assessor access (least privilege) . . . 9
- G3. Microsoft Graph / Az / management API access . . . 9
- G4. Licensing for the products in scope . . . 9
- G5. Scope definition and a point-in-time baseline . . . 9
- G6. Best-practice reference baseline . . . 9
- G7. Output handling and confidentiality . . . 10
- G8. Remediation ownership (for the paired remediation) . . . 10

3. Implementation — Security Optimization Assessment: Defender for Cloud Apps 10
- Stage 1 — Assess (define scope and capture the baseline) . . . 10
- Stage 2 — Configure (set the scoring reference) . . . 11
- Stage 3 — Pilot (score the posture on the defined scope) . . . 11
- Stage 4 — Validate (produce and review the findings report) . . . 12

Stage 5 — Expand (broaden scope and trend) . . . 12
5. Testing — Security Optimization Assessment: Defender for Cloud Apps 13
Test 1 — Read-only connection succeeds . . . 13
Test 2 — Discovery captured all four domains . . . 13
Test 3 — Control validation returns evidence, not just a verdict . . . 13
Test 4 — OAuth audit enumerates app grants . . . 13
Test 5 — Scoring is reproducible . . . 13
Test 6 — Findings report is prioritized and traceable . . . 13
Test 7 — Drift is detected across runs . . . 14
Test 8 — Output is stored securely . . . 14
Exit criteria . . . 14
6. Troubleshooting — Security Optimization Assessment: Defender for Cloud Apps 14
`Connect-MgGraph` fails or prompts for admin consent . . . 14
`SOAMCA-Discovery.ps1` returns empty connector/policy sections . . . 14
App governance section is absent . . . 15
OAuth audit lists nothing . . . 15
Scores differ between runs on the same baseline . . . 15
Report shows no drift when a prior run exists . . . 15
Throttling / partial results on large tenants . . . 15
Findings look wrong / stale versus the portal . . . 15
7. Best Practices — Security Optimization Assessment: Defender for Cloud Apps 15
Keep it read-only and provably so . . . 16
Score against a named, current baseline . . . 16
Make the score comparable across runs . . . 16
Prioritize by real risk, not by count . . . 16
Respect the decision boundary . . . 16
Handle the output as sensitive . . . 16
Re-run on a cadence . . . 16
Where to go next (adjacent catalog areas) . . . 17
8. Deliverables Summary — Security Optimization Assessment: Defender for Cloud Apps 17
Documentation . . . 17
PowerShell scripts (Microsoft Graph / Security — all read-only) . . . 17
What the buyer receives . . . 18
Outcome . . . 18
Appendix A — PowerShell Scripts 19

Executive Summary — Security Optimization Assessment: Defender for Cloud Apps

White-label · Assessments

A deployed product is not an optimized one. Microsoft Defender for Cloud Apps (the CASB in Microsoft 365) is frequently turned on, connected to one or two apps, and then left as-is — while its **app connectors**, **app-governance / OAuth policies**, **session and access (Conditional Access App Control) policies**, and **alert tuning** drift out of best practice. This **Security Optimization Assessment** takes a **read-only** look at exactly that configuration, scores it against Microsoft's recommended baseline, and hands you a **prioritized, risk-ranked findings report** telling you what to fix, in what order.

This package is written for an administrator with no prior assessment experience. It is **read-only by design** — every script here observes and reports; **none change your tenant**. The paired remediation work (closing the gaps) is a separate, pilot-first effort delivered by the **Remediation Playbook for Defender Security Optimization Assessments**.

What you get

- A plain-language **Solution Overview** of what Defender for Cloud Apps is, the four assessment domains (connector/discovery coverage, app-governance/OAuth, session/access, alert hygiene), and the scoring model.
- A **Requirements Guide** with verify / implement-if-missing for every solution-specific prerequisite (the category-wide prerequisites live once in the Global Requirements).
- An **Implementation Guide** of exact, ordered portal and PowerShell steps to run the assessment end to end — enumerate, score, and produce the findings report — following **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**.
- Five **PowerShell scripts** (Microsoft Graph / Security) — all **read-only** — for discovery, control validation, posture scoring, an OAuth/app-governance deep audit, and the prioritized findings report with drift versus a saved baseline.
- **Testing** and **Troubleshooting** guides so the run is repeatable and trustworthy.

What this delivers

A **scored, evidence-backed picture** of your Defender for Cloud Apps posture: which apps are connected and which discovery sources are feeding data, whether app governance and OAuth-app policies are catching risky consent, whether session/access policies enforce real-time controls, and whether alerts are tuned or drowning the analyst. The output is a **prioritized remediation backlog** — highest-risk gaps first — ready to hand to the Remediation Playbook.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — assess on a defined scope, produce and validate the findings, then expand the scope and re-run to trend the score.

Boundary

This assessment supplies **the evidence and the technical score**, not the authorization. **Risk-acceptance** of any finding, **remediation priority** in your context, and any **regulatory / records / incident-response** determination are the customer's security, compliance, and SOC functions' decisions. Licensing, plan, and preview facts are flagged **verify** — confirm current availability in your tenant.

Effort

A competent administrator can run the full assessment on a defined scope in a focused engagement (typically part of a day for discovery + scoring, plus review time for the findings). It is **repeatable** — re-run on a schedule to trend the posture score.

1. Solution Overview — Security Optimization Assessment: Defender for Cloud Apps

What Defender for Cloud Apps is

Microsoft Defender for Cloud Apps (formerly Microsoft Cloud App Security, "MCAS") is Microsoft's **cloud access security broker (CASB)**. It gives you four broad capabilities:

- **Cloud discovery** — ingests firewall/proxy/endpoint traffic logs (and signals from Defender for Endpoint) to surface **shadow IT** — the apps people actually use — and rank them by risk.
- **App connectors (API connectors)** — connect first- and third-party SaaS apps (Microsoft 365, Entra ID, and apps such as common file-sharing, CRM, and collaboration SaaS) over their APIs to pull activity, files, accounts, and OAuth grants.
- **App governance & OAuth-app policies** — monitor and govern the **OAuth applications** granted access to your data, flagging over-permissioned, unverified, or anomalous apps (app governance is an add-on capability within Defender for Cloud Apps).
- **Session & access policies (Conditional Access App Control)** — real-time controls that proxy sessions to **monitor or block** actions (download, copy, print, upload) inside SaaS apps, working with Entra Conditional Access.

On top of these it raises **alerts** (policy matches, anomaly detections, threat detections) that flow into the Microsoft Defender portal and Microsoft Defender XDR incidents.

Why this assessment matters

Most tenants have Defender for Cloud Apps **on** but not **optimized**: discovery is fed by one manual log upload instead of continuous ingestion; only Microsoft 365 is API-connected while the risky third-party SaaS is not; app governance is unlicensed or unconfigured so risky OAuth consent goes unseen; session policies are absent so there is no real-time control; and alerts are either noisy (never tuned) or silent (nobody owns them). None of this is visible from a glance at the portal — you need to **enumerate the configuration, compare it to the recommended baseline, and score it**.

That is exactly what this assessment does. It is **read-only** — it changes nothing — and it produces a **prioritized, risk-ranked findings report** across the four domains so remediation effort goes to the highest-risk gaps first.

The four assessment domains and how they are scored

1. **Connector & discovery coverage** — Are the core apps API-connected? Is cloud discovery fed **continuously** (automated log upload / Defender for Endpoint integration) rather than by one-off snapshots? Are sanctioned/unsanctioned tags applied?

2. **App-governance & OAuth policies** — Is app governance enabled? Are there policies flagging over-permissioned, unverified-publisher, or dormant/anomalous OAuth apps? Are highly privileged consented apps reviewed?

3. **Session & access policies** — Are Conditional Access App Control **session** and **access** policies in place for the apps that need real-time control (block download to unmanaged devices, block risky uploads)? Are they piloted before enforcement?

4. **Alert hygiene** — Are alert policies tuned (severity, scope, exclusions) so the signal is actionable? Is alert **resolution** happening (not a growing backlog of open alerts)? Are duplicate/legacy policies retired?

Each domain is scored against the **best-practice reference baseline** named in the Global Requirements (Microsoft security baseline / Secure Score guidance for Defender for Cloud Apps — *baselines evolve; verify against current Microsoft guidance*). The domain scores roll up into an overall posture score, and every gap becomes a finding with a **risk rank** (severity × exposure) so it can be prioritized.

What is in scope

• **Reading** Defender for Cloud Apps configuration: connectors, discovery sources, policies (app-governance/OAuth, session, access, activity, file, anomaly), and alert state.

• **Scoring** that configuration against the recommended baseline across the four domains.

• **Producing** a prioritized, risk-ranked findings report with a saved baseline for trending.

What is out of scope (and where it lives)

• **Making changes / closing the gaps** — that is the **Remediation Playbook for Defender Security Optimization Assessments** (pilot-first, `-WhatIf`-gated).

• **First-time onboarding** of Defender for Cloud Apps — see the *Onboarding Accelerator — Microsoft Defender for Cloud Apps* solution.

• **DLP for cloud apps / file policies design** — see the *Cloud App DLP with Defender for Cloud Apps* solution.

• **Cross-pillar (endpoint/identity/email/cloud) suite posture** — see the *Security Optimization Assessment — Microsoft Defender (Suite / XDR)* solution.

• **Regulatory, records, incident-response, and risk-acceptance decisions** — the customer's compliance/legal/SOC functions.

Connections and authentication used

• The **Microsoft Defender portal** (`security.microsoft.com`) → **Cloud apps** for connectors, discovery, policies, and alerts; **App governance** where licensed.

• **Microsoft Graph** (read-only Security scopes such as `SecurityEvents.Read.All`, and directory read scopes for OAuth-app enumeration such as `Application.Read.All` / `Directory.Read.All`) for the discovery, validation, scoring, OAuth-audit, and reporting scripts. *Verify current scope names and availability in your tenant.*

• Least-privilege roles: **Security Reader** / **Global Reader** are sufficient for this read-only assessment (per Global Requirements G2).

The standard deployment model

> ***Assess*** *the current Defender for Cloud Apps configuration (connectors, discovery, policies, alerts) →* ***Configure*** *the assessment scope and baseline reference →* ***Pilot*** *the run on the defined scope and review the findings →* ***Validate*** *the score and findings with the security owner →* ***Expand*** *the scope (more apps/users) and re-run to trend. Repeatable pattern:* ***Assess*** *→* ***Configure*** *→* ***Pilot*** *→* ***Validate*** *→* ***Expand****, entirely* ***read-only****.*

2. Requirements — Security Optimization Assessment: Defender for Cloud Apps

The category-wide prerequisites (the target tenant, read-only assessor access, Graph/Az access, licensing, scope/baseline, output handling, remediation ownership) are documented once in `global-requirements-assessments.md` — read that first. This section lists only the prerequisites **specific to assessing Defender for Cloud Apps**. Each item: **Description / Why required / How to verify / How to implement if missing.**

R1. Defender for Cloud Apps is provisioned in the tenant

Description: Defender for Cloud Apps (the cloud-apps area of the Microsoft Defender portal) is active for the tenant being assessed. **Why required:** There is no configuration to enumerate or score if the service has never been provisioned. **How to verify:** Sign in to `security.microsoft.com` → **Cloud apps** → **Cloud discovery**. If the area loads with a dashboard (rather than a "get started" splash), the service is active. **How to implement if missing:** Confirm a qualifying license is assigned (see R2) and open **Cloud apps** once to initialize the service. First-time onboarding is out of scope here — use the *Onboarding Accelerator — Microsoft Defender for Cloud Apps* solution. Record "service not provisioned" as a coverage finding rather than skipping silently.

R2. Licensing for Defender for Cloud Apps (and app governance)

Description: The licenses/plans that enable Defender for Cloud Apps and, separately, **app governance**. **Why required:** Some assessment domains only exist when the plan is enabled — e.g. app governance policies are only assessable when app governance is licensed and turned on. **How to verify:** In the Microsoft 365 admin center confirm the plan that includes Defender for Cloud Apps is assigned; in the Defender portal check whether **App governance** appears under **Cloud apps**. *License-to-feature mapping changes — verify current entitlements in your tenant.* **How to implement if missing:** Note any unlicensed capability as a **coverage gap finding** (a genuine posture gap), not an omission. Do not assume a feature is misconfigured when it is simply unlicensed — the finding is "license/enable to gain this control."

R3. Read-only Security Reader / Global Reader access

Description: A read-only role sufficient to enumerate Cloud Apps configuration and the OAuth apps in the directory. **Why required:** The assessment must prove it changes nothing; read-only roles are sufficient for every script here. **How to verify:** Entra admin center → **Roles and administrators** → confirm your account holds **Security Reader** and/or **Global Reader**. Confirm you can open **Cloud apps** → **Policies** and view (not edit) them. **How to implement if missing:** Assign **Security Reader** (or **Global Reader**). Do **not** grant write/admin roles for the assessment itself.

R4. Microsoft Graph read scopes consented

Description: The read-only Graph scopes the scripts use — Security read (e.g. `SecurityEvents.Read.All`) and directory/application read (e.g. `Application.Read.All`, `Directory.Read.All`) for OAuth-app enumeration. **Why required:** The discovery, validation, scoring, OAuth-audit, and reporting scripts collect configuration and posture through Graph. **How to verify:** Run `Connect-MgGraph -Scopes "SecurityEvents.Read.All","Application.Read.All","Directory.Read.All"` and confirm it succeeds; then `SOAMCA-Discovery.ps1` should run without a permissions error. *Verify current scope names and consent requirements.* **How to implement if missing:** Have an administrator consent the read-only scopes (delegated or app-only). Never request write scopes for this assessment.

R5. Cloud discovery data source context

Description: Knowledge of how (or whether) cloud discovery is being fed — automated log upload, Defender for Endpoint integration, or manual snapshot uploads. **Why required:** "Discovery coverage" is a scored domain; you must know the ingestion method to judge whether it is continuous versus one-off. **How to verify: Cloud apps** → **Cloud discovery** → **Settings** (data sources / automatic log upload) and **Settings** → **Microsoft Defender for Endpoint integration**. Note which are configured. **How to implement if missing:** Nothing to change for the assessment — simply **record the current ingestion method** so the scoring reflects it. Continuous ingestion scores higher than manual snapshots; no source at all is a finding.

R6. A defined assessment scope and a secure output location

Description: An agreed scope (which apps/users/connectors are in scope) and a secure place to store the findings report and the saved baseline. **Why required:** Scope keeps the score comparable across runs; the findings report is a map of your weaknesses and must be handled as sensitive (Global Requirements G5, G7). **How to verify:** The scope is written down; a restricted folder/location exists for `SOAMCA-*.json` outputs and the findings report. **How to implement if missing:** Agree the scope with the security owner before running; create a restricted output location. The scripts default to writing JSON outputs in the working directory — point them at the secure location.

Global Requirements — Assessments (category-level)

These requirements apply to every solution in the Assessments category (Security Optimization Assessments, remediation playbooks, and cross-cutting security assessments). Published once at the category level so listings stay small. Confirm these before running any assessment. Each entry:

Description / Why / Verify / Implement-if-missing.

> *An assessment observes and scores; it does not change your tenant. Every assessment script here is **read-only** (the paired remediation work is separate and pilot-first). Scope, risk-acceptance, and remediation-priority decisions belong to the customer's security/compliance owners — the assessment supplies the evidence, not the authorization.*

G1. The workload/tenant being assessed

Description: An active Microsoft 365 / Entra tenant and/or Azure subscription(s) hosting the products in scope. Why: There is nothing to assess without the target environment. Verify: Sign in to the relevant portal(s) (`security.microsoft.com`, `portal.azure.com`) as an administrator. Implement: Ensure the target tenant/subscription exists and the products under assessment are deployed (even partially — assessing coverage gaps is the point).

G2. Read-only assessor access (least privilege)

Description: A read-only role sufficient to enumerate configuration — e.g. Security Reader, Global Reader, and/or Reader at the Azure scope. Why: An assessment must never require write access; read-only proves it changes nothing. Verify: Entra -> `Roles and administrators`; confirm Security Reader / Global Reader (and Azure Reader where cloud resources are assessed). Implement: Assign the read-only roles; do not grant write/admin roles for the assessment itself.

G3. Microsoft Graph / Az / management API access

Description: Microsoft Graph (Security), Azure Resource Manager, and product-specific management APIs reachable with the assessor identity. Why: The assessment scripts collect configuration and posture via these APIs. Verify: `Connect-MgGraph` (read scopes) and `Connect-AzAccount` succeed; the assessment discovery script runs. *Verify current API/permission scopes.* Implement: Consent the read-only Graph scopes; ensure network access to the Microsoft endpoints.

G4. Licensing for the products in scope

Description: The licenses/plans for the products being assessed (e.g. Defender for Endpoint/Identity/Office 365/Cloud Apps plans, Defender for Cloud plans). Why: Some posture/configuration is only present (and only assessable) when the plan is enabled; the assessment flags plan gaps too. Verify: Confirm the relevant plans in the admin/billing portal. *Verify current plan/feature mapping.* Implement: Note unlicensed products as findings (a coverage gap) rather than skipping them silently.

G5. Scope definition and a point-in-time baseline

Description: A defined assessment scope (which products, subscriptions, and users) and the understanding that results are a point-in-time snapshot. Why: Scope keeps the assessment bounded and the score comparable; posture drifts, so results are dated. Verify: The scope is documented; the assessment records the run date. Implement: Agree the scope before running; re-run periodically to trend the score.

G6. Best-practice reference baseline

Description: The best-practice / benchmark the assessment scores against (Microsoft security baselines, Secure Score, CIS/Zero-Trust as applicable). Why: A score is only meaningful against a reference; the assessment states which baseline it uses. Verify: The reference baseline is named in the findings. *Baselines evolve — verify against current Microsoft guidance.* Implement: Use the documented baseline; note where organizational policy intentionally deviates.

G7. Output handling and confidentiality

Description: A place to store the findings report and an understanding that it may contain sensitive posture detail. Why: Assessment output is a roadmap of your weaknesses; it must be handled as sensitive. Verify: A secure destination for the report exists; access is restricted. Implement: Store findings securely; share on a need-to-know basis. *Risk-acceptance of any finding is the customer's decision.*

G8. Remediation ownership (for the paired remediation)

Description: Named owners for acting on findings (the remediation is separate, pilot-first work). Why: An assessment only creates value when someone owns closing the gaps. Verify: Remediation owners are identified; the Remediation Playbook solution is available for the fixes. Implement: Assign owners; use the Remediation Playbook (pilot-first, -WhatIf-gated) to close findings and re-score.

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation — Security Optimization Assessment: Defender for Cloud Apps

This is the exact, ordered procedure to run the assessment end to end. It is **read-only** — you enumerate configuration, score it, and produce a findings report; you change nothing in the tenant.

Naming convention. Every artifact you create in this run uses `<Purpose> – <Scope> – <Stage>`. Use these exact names in the scripts, testing, and the findings report:

- Assessment scope record → `SOA Scope – Cloud Apps – Assess`
- Baseline configuration snapshot → `Posture Baseline – Cloud Apps – Assess`
- Posture score record → `Posture Score – Cloud Apps – Pilot`
- Prioritized findings report → `Findings Report – Cloud Apps – Validate`

Placeholders use `contoso.onmicrosoft.com`. Substitute your tenant. Confirm the prerequisites in Section 2 and the Global Requirements before starting.

Stage 1 — Assess (define scope and capture the baseline)

Step 1.1 — Record the assessment scope

1. Open a working document named `SOA Scope – Cloud Apps – Assess`.
2. Record: the tenant (`contoso.onmicrosoft.com`), which apps/connectors are in scope, which user population, the **run date**, and the **best-practice reference baseline** you will score against (per

Global Requirements G6 — *verify against current Microsoft guidance*).

3. Confirm your account has **Security Reader**/**Global Reader** and the Graph read scopes (Section 2, R3/R4).

Step 1.2 — Sign in and confirm the service

1. Browse to `https://security.microsoft.com` and sign in.

2. Go to **Cloud apps** → **Cloud discovery**. Confirm the dashboard loads (service is provisioned — R1).

3. Note whether **App governance** appears under **Cloud apps** (licensing — R2).

Step 1.3 — Capture the configuration baseline (read-only)

1. Open PowerShell in the `scripts/` folder.

2. Connect read-only:

```
Connect-MgGraph -Scopes "SecurityEvents.Read.All","Application.Read.All","Directory.Read.All"
```

1. Run discovery and save the snapshot as the baseline:

```
.\SOAMCA-Discovery.ps1 -OutputPath ".\Posture Baseline - Cloud Apps - Assess.json"
```

1. This enumerates connectors, discovery data sources, the policy inventory (app-governance/OAuth, session, access, activity, file, anomaly), and alert state. It **makes no changes**. Keep the JSON as `Posture Baseline – Cloud Apps – Assess`.

Stage 2 — Configure (set the scoring reference)

Step 2.1 — Confirm the four assessment domains and weights

1. The scoring model covers: **connector & discovery coverage**, **app-governance & OAuth policies**, **session & access policies**, and **alert hygiene**.

2. Review the default weights in `SOAMCA-Score.ps1` (documented in its header). If your security owner wants a different emphasis (for example, weighting OAuth risk higher), record the change in `SOA Scope – Cloud Apps – Assess` so runs stay comparable.

Step 2.2 — Validate that best-practice controls are present (read-only)

1. Run the control validation, supplying the values you confirmed in the portal for the manual checks (app governance on/off, policy counts, discovery ingestion method, alert tuning):

```
.\SOAMCA-Validate.ps1 -BaselinePath ".\Posture Baseline - Cloud Apps - Assess.json" `
    -AppGovernanceEnabled $true -OAuthPolicyCount 2 -SessionPolicyCount 1 -AccessPolicyCount 1 `
    -DiscoveryIngestion continuous -AlertPoliciesTuned $true -OutputPath ".\SOAMCA-validation.json"
```

1. This checks, for each domain, whether the recommended controls exist (core apps connected; continuous discovery ingestion; app governance enabled with OAuth policies; at least one session and one access policy; alert policies tuned and being resolved). Each check returns **Pass / Gap** with the evidence — it does not fix anything. It writes `SOAMCA-validation.json`.

Stage 3 — Pilot (score the posture on the defined scope)

Step 3.1 — Deep-audit OAuth / app governance

1. Run the OAuth app audit:

```
.\SOAMCA-OAuthAudit.ps1 -OutputPath ".\oauth-audit.json"
```

1. This enumerates OAuth application grants and flags over-permissioned scopes, unverified publishers, and dormant high-privilege apps — the evidence behind the app-governance domain. Read-only.

Step 3.2 — Compute the posture score

1. Run the scorer against the validation output:

```
.\SOAMCA-Score.ps1 -ValidationPath ".\SOAMCA-validation.json" -OutputPath ".\Posture Score - Cloud Apps
```

1. This produces per-domain scores and an overall posture score, saved as `Posture Score – Cloud Apps – Pilot`. Record the run date. (Scoring is deterministic — the same input always yields the same score.)

Stage 4 — Validate (produce and review the findings report)

Step 4.1 — Generate the prioritized findings report

1. Run the report generator (add `-PriorReportPath` on re-runs to show drift):

```
.\SOAMCA-Report.ps1 -ValidationPath ".\SOAMCA-validation.json" -ScorePath ".\Posture Score - Cloud Apps
```

1. The report lists every finding with its **domain**, **severity**, **risk rank** (severity × exposure), the **evidence**, and a **pointer to the remediation** (which Remediation Playbook section closes it). It also compares to any prior saved baseline to show **drift**.

Step 4.2 — Review with the security owner

1. Walk through `Findings Report – Cloud Apps – Validate` with the named security owner.

2. The owner confirms **risk-acceptance** and **remediation priority** — these are their decisions, not the assessment's. Record any accepted-risk items with justification and expiry.

3. Store the report securely (Global Requirements G7) — it is sensitive.

Stage 5 — Expand (broaden scope and trend)

Step 5.1 — Widen the scope and re-run

1. Expand `SOA Scope – Cloud Apps – Assess` to include additional apps/users.

2. Re-run Steps 1.3 → 4.1. Because you kept the prior baseline, `SOAMCA-Report.ps1` will show the **drift** (improvement or regression) since the last run.

Step 5.2 — Hand off to remediation and schedule the re-run

1. Hand `Findings Report – Cloud Apps – Validate` to the **Remediation Playbook for Defender Security Optimization Assessments** to close the gaps pilot-first.

2. Schedule the assessment to re-run on a cadence (for example quarterly) so the posture score trends over time.

Everything above is read-only. If any script reports a write attempt or requests a write scope, stop — it should not, and that would be a defect.

5. Testing — Security Optimization Assessment: Defender for Cloud Apps

These procedures validate that the assessment **ran correctly and completely** — that the enumeration is trustworthy, the scoring is reproducible, and the findings report is sound. Because the assessment is read-only, "testing" here means proving the *evidence* is reliable, not that a change took effect. Each test has an explicit **pass/fail gate**. Reference the scripts by the names used in Section 3.

Test 1 — Read-only connection succeeds

Procedure: Run `Connect-MgGraph -Scopes "SecurityEvents.Read.All","Application.Read.All","Directory.Read.All"`, then `.\SOAMCA-Discovery.ps1`. **Pass:** The script completes and writes `Posture Baseline - Cloud Apps - Assess.json`; no write scope was requested. **Fail:** A permissions error, or any prompt for a write/`ReadWrite` scope — re-check R3/R4 and confirm you are using a read-only role.

Test 2 — Discovery captured all four domains

Procedure: Open `Posture Baseline - Cloud Apps - Assess.json`. **Pass:** It contains sections for **connectors/discovery**, **app-governance/OAuth policies**, **session/access policies**, and **alerts** — even if a section is empty (an empty section is itself a finding). **Fail:** A domain section is missing entirely (not empty — absent). Re-run discovery; check that the account can view **Cloud apps** → **Policies**.

Test 3 — Control validation returns evidence, not just a verdict

Procedure: Run `.\SOAMCA-Validate.ps1 -BaselinePath ".\Posture Baseline - Cloud Apps - Assess.json"`. **Pass:** Each domain check returns **Pass** or **Gap** *with the underlying evidence* (e.g. "app governance: not enabled", "session policies: 0 found"). **Fail:** A check returns a verdict with no evidence, or throws — inspect the baseline JSON for the expected section.

Test 4 — OAuth audit enumerates app grants

Procedure: Run `.\SOAMCA-OAuthAudit.ps1 -OutputPath ".\oauth-audit.json"`. **Pass:** The output lists OAuth applications with their granted scopes and flags (over-permissioned / unverified-publisher / dormant). If the tenant genuinely has no third-party OAuth grants, the output is an empty set with that note. **Fail:** The script cannot enumerate applications — confirm `Application.Read.All` / `Directory.Read.All` are consented (R4).

Test 5 — Scoring is reproducible

Procedure: Run `.\SOAMCA-Score.ps1` twice against the **same** baseline. **Pass:** Both runs produce the **same** per-domain and overall scores (scoring is deterministic for a fixed input). **Fail:** Scores differ for identical input — a defect; do not trust the score until reproducible.

Test 6 — Findings report is prioritized and traceable

Procedure: Run `.\SOAMCA-Report.ps1 ...` and open `Findings Report - Cloud Apps - Validate.json`. **Pass:** Findings are sorted by **risk rank** (highest first); each finding names its **domain**, **severity**, **evidence**, and a **remediation pointer** to the Remediation Playbook. **Fail:** Findings are unsorted, or a finding lacks evidence or a remediation pointer.

Test 7 — Drift is detected across runs

Procedure: After a second run, confirm `SOAMCA-Report.ps1` compares to the prior saved baseline. **Pass:** The report shows **new**, **resolved**, and **unchanged** findings versus the prior run (or notes "no prior baseline" on the first run). **Fail:** No drift comparison appears when a prior baseline exists — check the baseline path argument.

Test 8 — Output is stored securely

Procedure: Confirm the `Findings Report - Cloud Apps - Validate` and score/baseline JSON live in the restricted location agreed in R6. **Pass:** Files are in the secure destination with access restricted to the security owner and assessor. **Fail:** The report is in an open location — move it; it is a map of your weaknesses (Global Requirements G7).

Exit criteria

All eight tests pass, the security owner has reviewed `Findings Report – Cloud Apps – Validate`, and risk-acceptance/priority decisions are recorded. The backlog is ready for the Remediation Playbook.

6. Troubleshooting — Security Optimization Assessment: Defender for Cloud Apps

Common failures when running the assessment, in **symptom** → **cause** → **fix** form. All fixes stay read-only.

`Connect-MgGraph` fails or prompts for admin consent

Symptom: `Connect-MgGraph -Scopes ...` errors, or a consent screen requires an administrator. **Cause:** The read scopes (`SecurityEvents.Read.All`, `Application.Read.All`, `Directory.Read.All`) have not been consented in the tenant, or your account lacks them. **Fix:** Have an administrator grant admin consent for the read-only scopes once. Confirm your account holds **Security Reader** / **Global Reader** (R3). *Verify current scope names — Microsoft occasionally renames scopes.*

`SOAMCA-Discovery.ps1` returns empty connector/policy sections

Symptom: The baseline JSON has empty `connectors` or `policies` sections in a tenant you know uses Cloud Apps. **Cause 1:** The API surfaces only what the read role can see — an insufficiently privileged account. **Cause 2:** The tenant genuinely has no API connectors / policies configured (that is a real finding, not an error). **Fix:** Confirm you can view **Cloud apps** → **Policies** in the portal with the same account. If the portal shows items the script does not, re-check the Graph scopes. If the portal is also empty, record it as a **coverage finding**.

App governance section is absent

Symptom: Validation reports app governance "not available" rather than "not configured." **Cause:** App governance is a licensed add-on and is not enabled in the tenant (R2). **Fix:** This is expected when unlicensed. Record it as a **license/enable coverage finding** — "enable app governance to gain OAuth-app monitoring." Do not treat it as a misconfiguration.

OAuth audit lists nothing

Symptom: `SOAMCA-OAuthAudit.ps1` produces an empty set. **Cause 1:** No third-party OAuth applications are consented in the tenant (possible in a small/new tenant). **Cause 2:** `Application.Read.All` / `Directory.Read.All` not consented, so enumeration silently returns nothing. **Fix:** Verify the scopes are consented (R4). Cross-check in Entra → **Enterprise applications**. If the portal shows apps but the script does not, the scope is missing.

Scores differ between runs on the same baseline

Symptom: `SOAMCA-Score.ps1` gives different numbers for identical input. **Cause:** A changed weight file, or the score is being computed against a *fresh* discovery rather than the saved baseline. **Fix:** Always pass the **same** saved baseline path. Confirm the weights in the script header were not edited between runs. Scoring is deterministic for fixed input (Testing Test 5).

Report shows no drift when a prior run exists

Symptom: The findings report says "no prior baseline" although you ran it before. **Cause:** The prior baseline JSON was not passed, or its filename/path changed. **Fix:** Point `SOAMCA-Report.ps1` at the retained prior baseline. Keep baselines with stable, dated filenames in the secure location.

Throttling / partial results on large tenants

Symptom: Discovery or the OAuth audit is slow, times out, or returns partial pages. **Cause:** Microsoft Graph throttling on large directories. **Fix:** Re-run; the scripts page through results and honor retry-after. For very large tenants, scope the run (fewer users/apps at a time) and combine baselines — record the scoping in `SOA Scope – Cloud Apps – Assess`.

Findings look wrong / stale versus the portal

Symptom: A finding contradicts what you see live in the portal. **Cause:** Posture is a **point-in-time snapshot**; the baseline was captured earlier than your portal view, or a change was made after the run. **Fix:** Re-run discovery to refresh the baseline, then re-score. Always trust the dated baseline over memory (Global Requirements G5).

7. Best Practices — Security Optimization Assessment: Defender for Cloud Apps

Field-proven guidance for running this assessment well and keeping the result trustworthy over time.

Keep it read-only and provably so

• Run with **Security Reader / Global Reader** only. If a script ever requests a write or `ReadWrite` scope, treat it as a defect and stop — this assessment must be able to prove it changed nothing.

• Store the connection scopes you used alongside the findings report as evidence of read-only execution.

Score against a named, current baseline

• Always record **which** best-practice baseline you scored against and its date (Global Requirements G6). Microsoft's guidance for Defender for Cloud Apps evolves — *verify against current Microsoft guidance* each cycle.

• Where organizational policy intentionally deviates from the baseline, note it as an **accepted deviation** rather than letting it read as a gap forever.

Make the score comparable across runs

• Keep the **scoring weights** fixed between runs (in `SOAMCA-Score.ps1`). If the security owner changes a weight, record it in `SOA Scope – Cloud Apps – Assess` and note that scores before/after are not directly comparable.

• Re-use the saved baseline so `SOAMCA-Report.ps1` can show **drift** — trending posture is more valuable than a single snapshot.

Prioritize by real risk, not by count

• The report ranks by **severity × exposure**. Resist "fix the easy ones first"; an unverified, over-permissioned OAuth app with tenant-wide data access outranks a dozen cosmetic alert-tuning items.

• Treat **OAuth / app-governance** findings with particular care — consent-grant attacks are a common, high-impact path and app governance is easy to leave unlicensed/unconfigured.

Respect the decision boundary

• The assessment supplies **evidence and a technical score**. **Risk-acceptance**, **remediation priority**, and any **regulatory or incident-response** call belong to the customer's security, compliance, and SOC functions. Record accepted risks with justification and expiry.

Handle the output as sensitive

• The findings report is a roadmap of your weaknesses. Store it in the restricted location (R6 / Global Requirements G7) and share on a need-to-know basis.

Re-run on a cadence

• Posture drifts as apps are connected, users change, and policies age. Schedule the assessment (for example quarterly, and after any major SaaS onboarding) to keep the score current.

Where to go next (adjacent catalog areas)

• **Remediation Playbook for Defender Security Optimization Assessments** — closes the findings pilot-first, `-WhatIf`-gated, then re-scores to prove closure. This is the natural next step.

• **Onboarding Accelerator — Microsoft Defender for Cloud Apps** — if the assessment reveals the service is barely provisioned, start here.

• **Cloud App DLP with Defender for Cloud Apps** — for designing the file/DLP policies the assessment measures.

• **Security Optimization Assessment — Microsoft Defender (Suite / XDR)** — to assess the whole Defender suite (endpoint/identity/email/cloud-apps/cloud) and unified RBAC/incident posture, not just Cloud Apps.

• **Security Optimization Assessment — Defender for Cloud** — the sibling assessment for Azure/multicloud workload posture.

8. Deliverables Summary — Security Optimization Assessment: Defender for Cloud Apps

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box — read-only assessment that scores posture and prioritizes gaps
1. Solution Overview	What Defender for Cloud Apps is, the four assessment domains (connector/discovery, app-governance/OAuth, session/access, alert hygiene), the scoring model, scope, and connections used
2. Requirements Guide	Solution-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Assessments)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps to run the assessment end to end — Assess → Configure → Pilot → Validate → Expand — with every artifact named
5. Testing Guide	Eight validation tests with pass/fail gates proving the evidence is reliable
6. Troubleshooting Guide	Common run failures and fixes
7. Best Practices	Field-proven guidance and where to go next

PowerShell scripts (Microsoft Graph / Security — all read-only)

Script	What it does
`SOAMCA-Discovery.ps1`	Read-only discovery — enumerates app connectors, cloud-discovery data sources, the policy inventory (app-governance/OAuth, session, access, activity, file, anomaly), and alert state; saves the configuration baseline. Makes no changes.
`SOAMCA-Validate.ps1`	Checks, per domain, whether the recommended best-practice controls are present (core apps connected, continuous discovery, app governance + OAuth policies, session/access policies, tuned/resolved alerts), returning Pass/Gap with evidence.
`SOAMCA-Score.ps1`	Computes deterministic per-domain and overall posture scores against the reference baseline using documented weights.
`SOAMCA-OAuthAudit.ps1`	Read-only deep audit of OAuth application grants — flags over-permissioned scopes, unverified publishers, and dormant high-privilege apps (the app-governance evidence).
`SOAMCA-Report.ps1`	Produces the prioritized, risk-ranked findings report (domain, severity, evidence, remediation pointer) and shows drift versus a prior saved baseline.

What the buyer receives

- This solution as a **Word** document and **PDF**.
- All five **PowerShell scripts** (read-only, Microsoft Graph / Security) in a single ZIP.
- A repeatable process following **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — read-only, scored, and traceable, with findings ready to hand to the Remediation Playbook.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A scored, evidence-backed picture of your Defender for Cloud Apps posture across connector/discovery coverage, app-governance/OAuth, session/access, and alert hygiene — and a prioritized remediation backlog that tells you what to fix, in order. Read-only: it changes nothing, it tells you what to fix.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

SOAMCA-Discovery.ps1

Purpose: Read-only discovery for the "Security Optimization Assessment — Defender for Cloud Apps" solution. Enumerates, via Microsoft Graph (Security) and directory read APIs: app/API connectors, cloud-discovery data sources, the Cloud Apps policy inventory (app-governance/ OAuth, session, access, activity, file, anomaly), and current alert state. Writes a single JSON snapshot used as the assessment baseline (`Posture Baseline - Cloud Apps - Assess`). Makes NO changes to the tenant. HOW TO RUN Connect-MgGraph -Scopes "SecurityEvents.Read.All","Application.Read.All","Directory.Read.All" powershell -ExecutionPolicy Bypass -File .\SOAMCA-Discovery.ps1 ` -OutputPath ".\Posture Baseline - Cloud Apps - Assess.json" Requires the Microsoft.Graph module and a read-only role (Security Reader / Global Reader). EXPECTED RESULTS A JSON baseline with four domain sections (connectors/discovery, app-governance/OAuth, session/access, alerts) — the point-in-time evidence the rest of the assessment scores. .NOTES White-label. Read-only — reports configuration/identifiers only; makes no changes. Licensing/plan/preview facts are indicative — verify current availability in your tenant. Some Cloud Apps configuration is not fully exposed via Graph; where an area cannot be read programmatically the script records it for manual confirmation in the portal. > [CmdletBinding()] param([string]$OutputPath = ".\Posture Baseline - Cloud Apps - Assess.json")

How to run: Connect-MgGraph -Scopes "SecurityEvents.Read.All","Application.Read.All","Directory.Read.All" powershell -ExecutionPolicy Bypass -File .\SOAMCA-Discovery.ps1 ` -OutputPath ".\Posture Baseline - Cloud Apps - Assess.json" Requires the Microsoft.Graph module and a read-only role (Security Reader / Global Reader). EXPECTED RESULTS A JSON baseline with four domain sections (connectors/discovery, app-governance/OAuth, session/access, alerts) — the point-in-time evidence the rest of the assessment scores. .NOTES White-label. Read-only — reports configuration/identifiers only; makes no changes. Licensing/plan/preview facts are indicative — verify current availability in your tenant. Some Cloud Apps configuration is not fully exposed via Graph; where an area cannot be read programmatically the script records it for manual confirmation in the portal. > [CmdletBinding()] param([string]$OutputPath = ".\Posture Baseline - Cloud Apps - Assess.json")

SOAMCA-OAuthAudit.ps1

Purpose: Enumerates the OAuth applications (service principals + delegated/app-role grants) in the tenant and flags the risk indicators that app governance is meant to catch: - Over-permissioned: broad/tenant-wide application permissions (e.g. *.ReadWrite.All) - Unverified publisher: no verified publisher / unknown publisher - Dormant high-privilege: privileged grants on apps with no recent sign-in signal Produces the detailed evidence behind the app-governance/OAuth assessment domain. NO changes. HOW TO RUN Connect-MgGraph -Scopes "Application.Read.All","Directory.Read.All" powershell -ExecutionPolicy Bypass -File .\SOAMCA-OAuthAudit.ps1 -OutputPath ".\oauth-audit.json" EXPECTED RESULTS A JSON list of OAuth apps with granted scopes and risk flags, sorted by risk. An empty set (with a note) is valid in a tenant with no third-party OAuth grants. .NOTES White-label. Read-only. Flags are heuristic evidence for a human reviewer, NOT an automatic verdict — the security owner decides risk-acceptance. Verify current Graph scope names. > [CmdletBinding()] param([string]$OutputPath = ".\oauth-audit.json")

How to run: Connect-MgGraph -Scopes "Application.Read.All","Directory.Read.All" powershell -ExecutionPolicy Bypass -File .\SOAMCA-OAuthAudit.ps1 -OutputPath ".\oauth-audit.json" EXPECTED RESULTS A JSON list of OAuth apps with granted scopes and risk flags, sorted by risk. An empty set (with a note) is valid in a tenant with no third-party OAuth grants. .NOTES White-label. Read-only. Flags are heuristic evidence for a human reviewer, NOT an automatic verdict — the security owner decides risk-acceptance. Verify current Graph scope names. > [CmdletBinding()] param([string]$OutputPath = ".\oauth-audit.json")

SOAMCA-Report.ps1

Purpose: Combines the validation output (Pass/Gap per control), the posture score, and the OAuth audit into a single prioritized findings report. Each finding carries: domain, severity, risk rank (severity x exposure), evidence, and a remediation pointer to the Remediation Playbook. If a prior report/baseline is supplied, shows new / resolved / unchanged findings (drift). NO changes are made. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SOAMCA-Report.ps1 `-ValidationPath ".\SOAMCA-validation.json"` -ScorePath ".\Posture Score - Cloud Apps - Pilot.json" `-OAuthAuditPath ".\oauth-audit.json"` -PriorReportPath ".\Findings Report - Cloud Apps - Validate (previous).json" ` -OutputPath ".\Findings Report - Cloud Apps - Validate.json" -PriorReportPath is optional (omit on the first run). EXPECTED RESULTS A findings report (JSON + console summary) sorted by risk rank, with drift when a prior run is supplied. This is the backlog handed to the Remediation Playbook. .NOTES White-label. Read-only. Risk-acceptance and remediation priority are the security owner's decisions — this report supplies the evidence and a suggested rank, not the authorization. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ValidationPath, [Parameter(Mandatory)][string]$ScorePath, [string]$OAuthAuditPath, [string]$PriorReportPath, [string]$OutputPath = ".\Findings Report - Cloud Apps - Validate.json")

How to run: powershell -ExecutionPolicy Bypass -File .\SOAMCA-Report.ps1 `-ValidationPath ".\SOAMCA-validation.json"` -ScorePath ".\Posture Score - Cloud Apps - Pilot.json" `-OAuthAuditPath ".\oauth-audit.json"` -PriorReportPath ".\Findings Report - Cloud Apps - Validate (previous).json" ` -OutputPath ".\Findings Report - Cloud Apps - Validate.json" -PriorReportPath is optional (omit on the first run). EXPECTED RESULTS A findings report (JSON + console summary) sorted by risk rank, with drift when a prior run is supplied. This is the backlog handed to the Remediation Playbook. .NOTES White-label. Read-only. Risk-acceptance and remediation priority are the security owner's decisions — this report supplies the evidence and a suggested rank, not the authorization. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ValidationPath, [Parameter(Mandatory)][string]$ScorePath, [string]$OAuthAuditPath, [string]$PriorReportPath, [string]$OutputPath = ".\Findings Report - Cloud Apps - Validate.json")

SOAMCA-Score.ps1

Purpose: Consumes the validation output (SOAMCA-Validate.ps1) and computes a posture score for each of the four assessment domains and an overall weighted score (0-100). Scoring is DETERMINISTIC for a fixed input so runs are comparable. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SOAMCA-Score.ps1 `-ValidationPath ".\SOAMCA-validation.json"` -OutputPath ".\Posture Score - Cloud Apps - Pilot.json" EXPECTED RESULTS A JSON score record: per-domain score, weight, and overall score, plus the run date. SCORING MODEL (default weights — edit here to re-weight; record the change in SOA Scope): connectorsDiscovery = 25 (coverage is the foundation) appGovernanceOAuth = 30 (consent-grant risk is high-impact) sessionAccessPolicies = 25 (real-time control) alertHygiene = 20 (actionable signal) Domain score = (passed controls / total controls in domain) *

100. Overall = weighted average of domain scores. .NOTES White-label. Read-only. The baseline this scores against evolves — verify current Microsoft guidance for Defender for Cloud Apps each cycle. Weights are a starting point, not a mandate. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ValidationPath, [string]$OutputPath = ".\Posture Score - Cloud Apps - Pilot.json")

How to run: powershell -ExecutionPolicy Bypass -File .\SOAMCA-Score.ps1 `-ValidationPath ".\SOAMCA-validation.json"` -OutputPath ".\Posture Score - Cloud Apps - Pilot.json" EXPECTED RESULTS A JSON score record: per-domain score, weight, and overall score, plus the run date. SCORING MODEL (default weights — edit here to re-weight; record the change in SOA Scope): connectorsDiscovery = 25 (coverage is the foundation) appGovernanceOAuth = 30 (consent-grant risk is high-impact) sessionAccessPolicies = 25 (real-time control) alertHygiene = 20 (actionable signal) Domain score = (passed controls / total controls in domain) * 100. Overall = weighted average of domain scores. .NOTES White-label. Read-only. The baseline this scores against evolves — verify current Microsoft guidance for Defender for Cloud Apps each cycle. Weights are a starting point, not a mandate. > [CmdletBinding()] param([Parameter(Mandatory)][string]$ValidationPath, [string]$OutputPath = ".\Posture Score - Cloud Apps - Pilot.json")

SOAMCA-Validate.ps1

Purpose: Consumes the baseline produced by SOAMCA-Discovery.ps1 (plus the operator's portal-confirmed values) and evaluates, per domain, whether the recommended controls exist: - Connector & discovery coverage: core apps connected + continuous discovery ingestion - App-governance & OAuth: app governance enabled + OAuth-app policies present - Session & access: at least one session and one access policy, piloted before enforce - Alert hygiene: alert policies tuned + open backlog under control Each check returns Pass or Gap WITH the evidence. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\SOAMCA-Validate.ps1 ` -BaselinePath ".\Posture Baseline - Cloud Apps - Assess.json" Optional portal-confirmed inputs (from the manual checks) can be supplied as parameters below. EXPECTED RESULTS A per-domain Pass/Gap table with evidence — the input to SOAMCA-Score.ps1 and the report. .NOTES White-label. Read-only. Reflects a point-in-time snapshot. Licensing/preview facts flagged verify. > [CmdletBinding()] param([Parameter(Mandatory)][string]$BaselinePath, Portal-confirmed values the operator records from the manual checks (read-only observations): [bool]$AppGovernanceEnabled = $false, [int]$OAuthPolicyCount = 0, [int]$SessionPolicyCount = 0, [int]$AccessPolicyCount = 0, [ValidateSet('continuous','manual','none')][string]$DiscoveryIngestion = 'none', [bool]$AlertPoliciesTuned = $false, [string]$OutputPath = ".\SOAMCA-validation.json")

How to run: powershell -ExecutionPolicy Bypass -File .\SOAMCA-Validate.ps1 ` -BaselinePath ".\Posture Baseline - Cloud Apps - Assess.json" Optional portal-confirmed inputs (from the manual checks) can be supplied as parameters below. EXPECTED RESULTS A per-domain Pass/Gap table with evidence — the input to SOAMCA-Score.ps1 and the report. .NOTES White-label. Read-only. Reflects a point-in-time snapshot. Licensing/preview facts flagged verify. > [CmdletBinding()] param([Parameter(Mandatory)][string]$BaselinePath, Portal-confirmed values the operator records from the manual checks (read-only observations): [bool]$AppGovernanceEnabled = $false, [int]$OAuthPolicyCount = 0, [int]$SessionPolicyCount = 0, [int]$AccessPolicyCount = 0, [ValidateSet('continuous','manual','none')][string]$DiscoveryIngestion = 'none', [bool]$AlertPoliciesTuned = $false, [string]$OutputPath = ".\SOAMCA-validation.json")